

WORLD ROBOT OLYMPIAD 2026
FUTURE ENGINEERS CATEGORY

Engineering Documentation

Autonomous Obstacle-Avoiding Vehicle

Submission details	
Team name	YO_CNTRL_ALT_DEFEAT
Team ID	1684
Team members names	Abhay, Vikrant, Nidhella
Country / region	INDIA
Season	2026 — Future Engineers
Repository URL	https://github.com/SH047/Yo_Cntrl_Alt_Defeat
Document version	v1.0
Date	12/08/2026

Contents

1. System overview

Concept, architecture, division of responsibilities, hardware inventory

2. Mobility and mechanical design

Sub-systems, steering and drive configuration, torque and speed reasoning

3. Power and sensor architecture

Power tree, current strategy, sensor selection, placement, calibration, wiring

4. Software architecture and obstacle strategy

Master–slave split, modules, protocol, avoidance logic, state machine, testing

5. Systems thinking and engineering decisions

Subsystem interactions, constraints, trade-off register, risk register

6. Reproducibility and repository

Repository structure, README requirements, build and bring-up procedure

A. Appendix A — Wiring quick reference

Connection-by-connection build sheet

How this document maps to the judging rubric

Each rubric criterion is addressed by a dedicated section. Cross-references are given where a topic is naturally discussed in more than one place.

Rubric criterion	Addressed in	Principal evidence
1 — Mobility and mechanical design	Section 2	Sub-system table, steering/drive configuration, torque–speed strategy, Figures 2 and 3
2 — Power and sensor architecture	Section 3, Appendix A	Power tree, current strategy, sensor selection and placement, calibration method, wiring diagrams, Figures 4–6
3 — Software architecture and obstacle strategy	Section 4	Module decomposition, command protocol, avoidance logic, state machine, edge cases, Figures 7 and 8
4 — Systems thinking and engineering decisions	Section 5	Subsystem interaction map, constraint list, trade-off register, risk register, Figure 9
5 — Reproducibility and repository quality	Section 6, Appendix B	Repository structure, README requirements, bring-up procedure, completion checklist

1. System overview

1.1 Concept

The robot is a rear-wheel-drive autonomous vehicle with front-wheel steering, three ultrasonic distance sensors and Raspberry Pi computer vision. The Raspberry Pi uses a camera and OpenCV to detect obstacles and colours, and sends high-level commands such as `dodgeRight()` and `dodgeLeft()` to an ESP32 over a serial/UART link. The ESP32 handles ultrasonic sensing, steering and motor control.

The Raspberry Pi therefore performs high-level visual processing, while the ESP32 is responsible for real-time interaction with the physical hardware.

1.2 System architecture

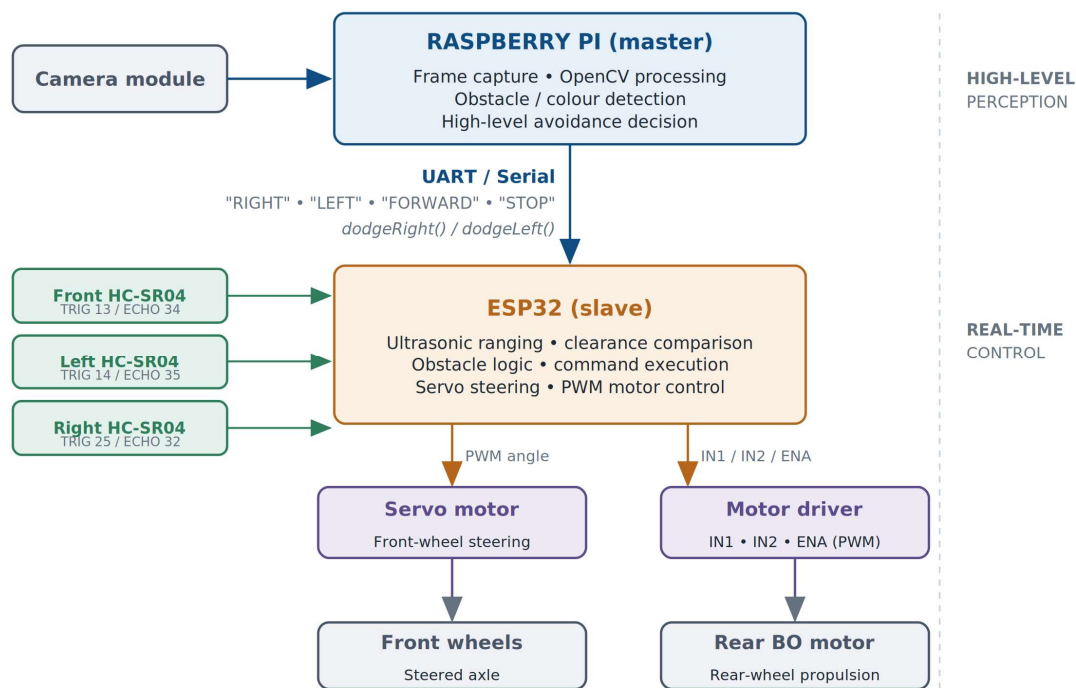


Figure 1. System architecture. Perception and decision-making run on the Raspberry Pi; all real-time sensing and actuation run on the ESP32.

1.3 Division of responsibilities

The two controllers have separate and non-overlapping responsibilities. This keeps computationally intensive vision processing on the Raspberry Pi while keeping time-sensitive hardware control on the ESP32.

Raspberry Pi — high-level perception	ESP32 — low-level sensing and actuation
Captures camera images	Reads the three ultrasonic sensors
Runs OpenCV	Determines local obstacle clearance
Detects obstacles and colours	Controls the motor driver
Makes high-level vision decisions	Controls the steering servo
Sends avoidance commands over UART	Executes commands received from the Raspberry Pi

1.4 Hardware inventory

Item	Qty	Role in the robot
Raspberry Pi	1	Vision host; runs OpenCV and issues avoidance commands
Raspberry Pi camera module	1	Image capture for the vision subsystem
ESP32 development board	1	Real-time controller for sensors, servo and drive motor
HC-SR04 ultrasonic sensor	3	Front, left and right distance measurement
Servo motor	1	Front-wheel steering actuation
BO motor	1	Rear-wheel propulsion
Single-channel motor driver with IN1, IN2 and ENA/PWM inputs	1	Direction and speed control of the BO motor
9 V battery / supply	1	Primary energy source
9 V → 5 V buck converter	1	Generates the 5 V rail for the servo and sensors
Front steering mechanism and chassis	1	Structure and steering linkage
Rear wheels coupled to the BO motor	2	Traction
1 kΩ and 2 kΩ resistors	3 pairs	ECHO voltage dividers (5 V → 3.3 V), one per sensor
Jumper wires and power wiring	as required	Interconnection

2. Mobility and mechanical design

Rubric criterion 1 This section addresses rubric criterion 1: chassis design choices, steering and drive mechanism justification, torque/speed reasoning and mechanical stability.

2.1 Sub-systems of the robot

The robot is divided into the following major sub-systems.

Sub-system	Components used	Function
Mechanical / locomotion	Rear drive wheels, front steering mechanism, chassis	Provides movement and directional control
Drive and motor control	DC (BO) motor, motor driver, ESP32	Controls the speed and direction of the rear drive motor
Steering	Servo motor, front steering mechanism	Changes the direction of the front wheels to steer the robot
Power supply	9 V battery/supply, buck converter (9 V → 5 V)	Provides suitable voltages to the different electrical components
Obstacle detection	Three ultrasonic sensors at the front, left and right	Measures the distance to nearby obstacles
Vision	Raspberry Pi, camera module, OpenCV	Detects and identifies obstacles using image processing
Communication / control	Raspberry Pi and ESP32 serial link	Transfers obstacle information and directions from the vision system to the motor-control system
Central control	ESP32	Processes sensor inputs and commands, and controls the motor and steering actions
Obstacle avoidance	dodgeRight() and dodgeLeft() routines, steering and drive mechanisms	Determines and executes the appropriate manoeuvre to avoid an obstacle

How these sub-systems interact as a whole is described in Section 5.1.

2.2 Steering and rear-wheel driving configuration

The robot uses a front-wheel steering and rear-wheel driving configuration, similar to the basic arrangement of a conventional vehicle. The front wheel is connected to a servo motor, allowing the steering angle to be adjusted, while the rear BO motor provides the driving force.

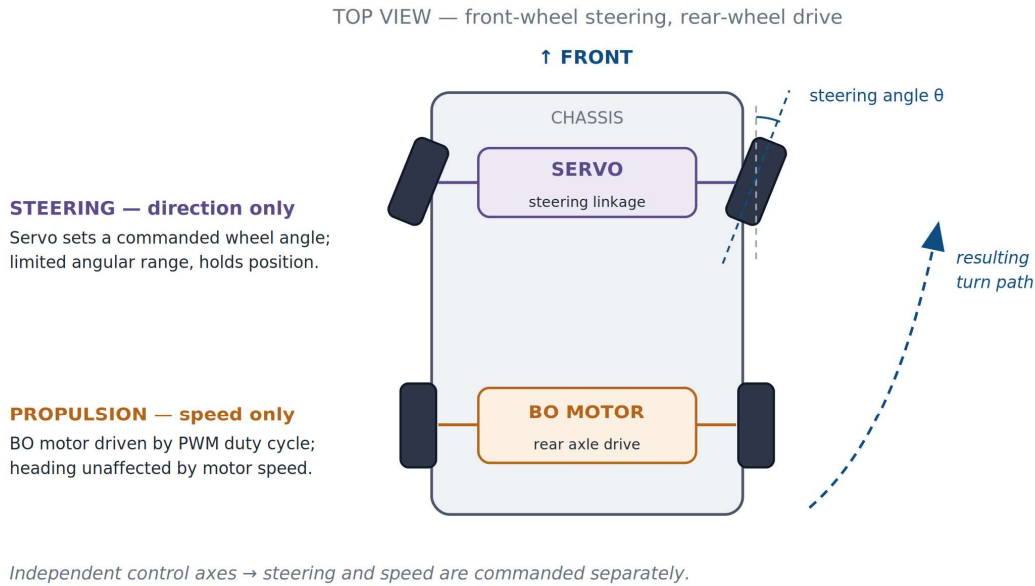


Figure 2. Front-wheel steering with rear-wheel drive. Steering angle and drive speed are independent control axes.

This arrangement provides a clear separation between directional control and propulsion. The servo changes the robot's heading without changing the speed of the rear wheels, while the rear motor continuously supplies the force required to move the robot forward. This makes the control system relatively simple, because steering and motion can be controlled independently.

The arrangement also provides good manoeuvrability for obstacle avoidance. When an obstacle is detected, the steered wheel can be turned toward the side with greater available clearance while the rear motor continues to propel the robot. This produces a smooth turning motion rather than requiring independent control of multiple drive wheels.

The steering servo also provides sufficient torque to change the direction of the front wheel against the mechanical resistance from the robot's weight and its contact with the ground.

2.3 Rear-wheel drive through a motor driver

A rear-wheel-drive arrangement provides the robot's main propulsion, while the front wheels are dedicated to steering. The motor driver is necessary because an ESP32 GPIO pin cannot directly supply the current required by a DC motor; it also allows the ESP32 to control both the direction and the speed of the motor.

Why a motor driver rather than the alternatives:

- A motor driver provides separation between the low-power control electronics and the higher-current motor circuit.
- PWM control can be used to vary motor speed.
- Forward and reverse operation can be implemented by changing the motor-driver inputs.

- Using a separate driver is more practical than attempting to drive the motor directly from the ESP32, which could damage the microcontroller.
- Compared with a more complex motor-control system, a conventional motor driver is simple, inexpensive and appropriate for a small mobile robot.

2.4 Front-wheel steering using a servo motor

The front wheels use a servo motor to control the steering angle. This allows the ESP32 to specify a particular steering position rather than simply turning a motor on or off.

Why a servo rather than the alternatives:

- A servo provides relatively precise angular positioning.
- It can maintain the required steering angle without requiring a separate position sensor in a typical setup.
- It is simpler to interface with the ESP32 than designing a steering mechanism using a DC motor and a feedback encoder.
- Compared with differential steering, front-wheel steering is more representative of conventional vehicle steering and can produce smoother directional changes.
- A servo is suitable when the steering mechanism only requires a limited angular range rather than continuous rotation.

2.5 Torque, speed and the variable-speed strategy

Variable speed control is important because the robot does not require the same speed in every situation. When the path is clear, the motor can operate at a higher PWM duty cycle, allowing the robot to travel faster. As an obstacle becomes closer, the motor speed can be reduced, giving the steering system more time to respond and reducing the distance required to slow down or stop.

The relationship between motor torque and speed matters here. For a DC BO motor, maximum torque is available at or near zero speed, while available torque generally decreases as motor speed increases. Reducing the commanded speed can therefore allow the motor to operate in a region where it can provide greater torque for a given load.

This is particularly beneficial during obstacle avoidance, because turning the steered wheel and moving the robot through a turn creates additional mechanical resistance. A lower speed gives better control and allows the motor to maintain sufficient torque while the robot is turning.

Situation	Commanded drive	Engineering reason
Obstacle far away	High PWM duty	Path is clear; travel time is minimised
Obstacle approaching	Reduce PWM duty	Gives the steering system more time to respond and shortens the distance needed to slow down

Situation	Commanded drive	Engineering reason
Obstacle close	Low duty or stop	Shortest stopping distance; the motor sits nearer the high-torque part of its curve
Steering around the obstacle	Low duty maintained	Turning adds mechanical resistance, so torque margin is prioritised over speed
Obstacle cleared	Increase PWM duty	Return to cruise

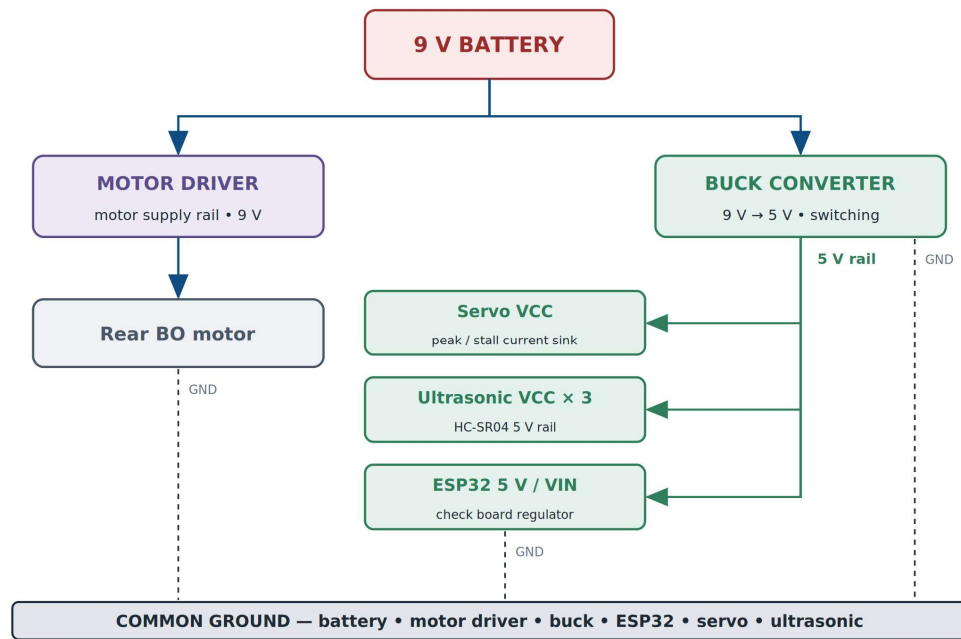
Overall, variable-speed control improves both safety and manoeuvrability, and also helps the drive motor maintain useful torque when the robot is subjected to greater load during steering and obstacle avoidance.

3. Power and sensor architecture

Rubric criterion 2 This section addresses rubric criterion 2: power system architecture, current-draw reasoning, sensor selection justification, sensor placement and calibration, and wiring diagrams.

3.1 Power system architecture

The 9 V supply powers the motor circuit and is also reduced to 5 V for the servo and the ultrasonic sensors. Keeping the motor supply on the raw battery rail means that motor current transients are not drawn through the same regulator that feeds the sensing and control electronics.



Every return path is tied to a single ground node so that control signals share one voltage reference.

Figure 3. Power distribution. The motor branch is taken directly from the battery; the buck converter feeds the 5 V logic and actuator rail.

The ESP32 may be supplied through its appropriate 5 V / VIN input if the board and regulator specifications permit it. The exact ESP32 development board must be checked before connecting power.

3.2 Common ground

All grounds must be connected together: battery ground, motor-driver ground, buck-converter ground, ESP32 ground, servo ground and ultrasonic ground. A common ground is essential because the ESP32 control signals need the same voltage reference as the devices receiving those signals.

Failure point A missing common ground is one of the most common causes of erratic behaviour in a mixed-voltage robot: signals appear to be present but are not interpreted correctly, because the receiving device measures them against a different reference.

3.3 Buck converter for the 5 V rail

A buck converter reduces the approximately 9 V supply to 5 V for the servo motor and the ultrasonic sensors.

Why a buck converter rather than the alternatives:

- A servo normally requires a lower voltage than the main supply, so a direct connection to 9 V could damage it.
- A buck converter is more efficient than a linear regulator, because it dissipates substantially less power as heat when reducing voltage.
- It allows the existing higher-voltage supply to be converted to the required servo voltage without requiring a separate battery.
- It is particularly useful in a mobile robot, because power efficiency directly affects operating time.

3.4 Current strategy and power budget

Loads are allocated to rails according to how much current they draw and how abruptly that current changes. The drive motor is the largest and most abrupt load, so it is kept on the battery rail. The servo is the largest load on the 5 V rail because its current rises sharply when it steers against mechanical resistance; for this reason it is powered from the buck converter and never from an ESP32 GPIO pin.

Load	Rail	Load character	Indicative draw	Measured
BO motor (through the driver)	9 V	Large, abrupt; highest at stall and on direction change	Motor- and load-dependent	
Steering servo	5 V (buck)	Low when idle, high peaks while steering against resistance	Servo-dependent; peak dominates sizing	
HC-SR04 × 3	5 V (buck)	Small and steady while ranging	A few tens of mA in total	
ESP32 board	5 V / VIN	Moderate and continuous; rises when radios are active	Board-dependent	
Raspberry Pi	Not yet allocated	Largest continuous logic load	See note below	

► *Measure each load with a multimeter in series (and note the peak while the servo stalls and while the motor starts). Enter the measured values in the last column, then confirm that the sum of the 5 V loads is within the buck converter's rated output current.*

Open item The power tree above does not yet define a supply for the Raspberry Pi. Because the Pi's continuous current requirement is significantly higher than the other 5 V loads, this must be resolved before the final build — either by sizing the buck converter to carry it or by giving the Pi its own regulated supply. The decision, once made, belongs in Section 3.1.

3.5 Sensor selection: three ultrasonic sensors

Three ultrasonic sensors are positioned at the front, left and right of the robot. They provide distance measurements around the robot rather than relying on information from a single direction.

Why ultrasonic sensors rather than the alternatives:

- Ultrasonic sensors directly measure the distance between the robot and an obstacle.
- They work independently of visible light, so their operation is less dependent on lighting conditions than camera-based distance estimation.
- They are relatively inexpensive and straightforward to interface with the ESP32.
- Sensors on the front, left and right give the robot information about possible escape directions.
- Infrared sensors could also be used, but their measurements are more affected by the reflectivity and surface characteristics of the obstacle.
- A LiDAR system could provide more detailed spatial information, but it would be more expensive and is unnecessary for basic obstacle-distance detection.

The ultrasonic sensors therefore provide a non-visual distance-sensing layer that complements the camera.

3.6 Sensor placement and coverage

The three sensors are mounted at the front, front-left and front-right. The front sensor primarily detects obstacles directly in the robot's path; the left and right sensors determine whether there is more free space on either side. Because the robot must steer before it reaches an obstacle, the side sensors are angled forward rather than square to the body, so that they measure the clearance the robot would actually turn into.

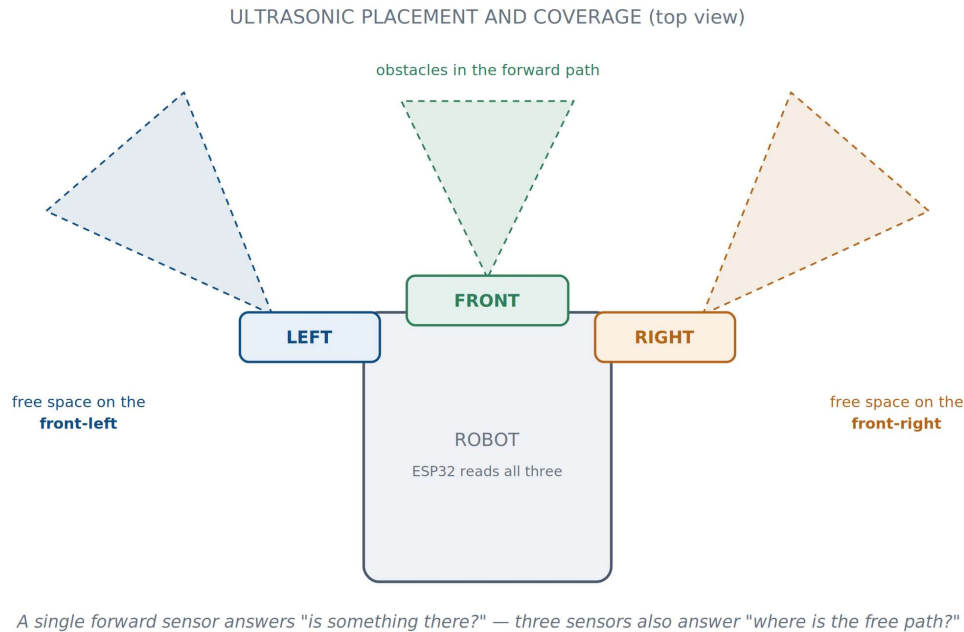


Figure 4. Sensor placement and coverage. Forward-angled side sensors measure the clearance available in each candidate escape direction.

This arrangement provides considerably more information than a single ultrasonic sensor would:

Front	Left	Right	Possible response
Close	Far	Far	Reduce speed / turn
Close	Close	Far	Steer right
Close	Far	Close	Steer left
Close	Close	Close	Stop and reassess
Far	Far	Far	Continue forward

The three-sensor arrangement is therefore useful because obstacle avoidance requires not only detecting whether an obstacle exists, but also determining where the available path is. The sensors are handled by the ESP32, which allows distance measurements and steering/motor decisions to be made locally without requiring the Raspberry Pi to process every ultrasonic measurement.

3.7 Sensor calibration and threshold selection

Each HC-SR04 returns an echo pulse whose width is proportional to the round-trip time of flight. Distance is recovered from the pulse width using the speed of sound: the measured time is halved to account for the outbound and return path, then multiplied by the speed of sound (approximately 0.0343 cm/ μ s at room temperature). Because the speed of sound varies with temperature, and because each module has its own small offset, every sensor is calibrated individually before the thresholds are set.

Calibration procedure:

1. Mount the sensor in its final position on the robot — calibrating a sensor off the vehicle does not capture mounting angle or reflections from the chassis.
2. Place a flat target square to the sensor at a known distance and record the value reported by the firmware.
3. Repeat at several distances across the working range, and repeat each measurement a few times to see the spread.
4. Record the difference between the true and reported distance; if there is a consistent offset, apply it as a per-sensor correction in firmware.
5. Find the minimum reliable distance (below which the module stops returning a usable echo) and the maximum usable distance, and treat readings outside that band as invalid.

3.8 ECHO level adaptation

The HC-SR04 ECHO output can be approximately 5 V, while the ESP32 GPIO pins are designed for 3.3 V logic. A 5 V ECHO signal must not be connected directly to an ESP32 input. One voltage divider is used on every ECHO line.

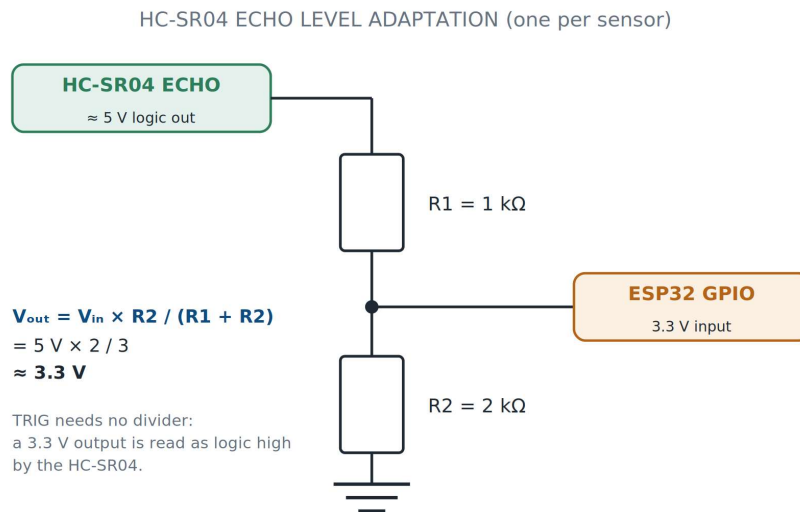


Figure 5. ECHO voltage divider. One divider is built for each of the three sensors.

A 1 kΩ upper resistor and a 2 kΩ lower resistor produce approximately 3.3 V from a 5 V ECHO signal. Three identical dividers are required: front ECHO to GPIO 34, left ECHO to GPIO 35, and right ECHO to GPIO 32. The TRIG lines need no divider, because the 3.3 V output of the ESP32 is read as a logic high by the HC-SR04.

3.9 ESP32 pin assignment

Component	Connection	ESP32 pin
Front ultrasonic	TRIG	GPIO 13
Front ultrasonic	ECHO (through divider)	GPIO 34
Left ultrasonic	TRIG	GPIO 14
Left ultrasonic	ECHO (through divider)	GPIO 35
Right ultrasonic	TRIG	GPIO 25
Right ultrasonic	ECHO (through divider)	GPIO 32
Motor driver IN1	Direction control	GPIO 18
Motor driver IN2	Direction control	GPIO 19
Motor driver ENA	Speed / PWM	GPIO 15
Servo signal	PWM	Free suitable GPIO — record the pin used
Common ground	GND	ESP32 GND

Note GPIO 34 and GPIO 35 are input-only pins on the ESP32, which is appropriate for the ultrasonic ECHO signals but means they cannot be reassigned to drive anything.

3.10 Servo wiring

The servo controls the front steering mechanism. Its supply comes from the buck converter, and only its signal line is connected to the ESP32:

```
Buck +5 V   → Servo VCC
Buck GND    → Servo GND
ESP32 GPIO → Servo SIGNAL
```

The servo must not be powered from an ESP32 GPIO pin. Steering against mechanical resistance can cause the servo to draw significant current, so the 5 V buck converter supplies it.

3.11 Motor driver wiring

The ESP32 controls the rear BO motor through the motor driver:

```
ESP32 GPIO 18 → Motor driver IN1
ESP32 GPIO 19 → Motor driver IN2
ESP32 GPIO 15 → Motor driver ENA / PWM
ESP32 GND     → Motor driver GND

9 V battery   → Motor driver motor-power input
```

```
Motor driver OUT1 → BO motor
Motor driver OUT2 → BO motor
```

The exact power and output terminals depend on the motor-driver module, so its pin labels and voltage/current ratings must be verified against its datasheet before the motor is connected.

Corresponding firmware definitions:

```
#define MOTOR_PIN_1 18
#define MOTOR_PIN_2 19
#define ENA          15
```

3.12 Vision sensor: Raspberry Pi and camera

The camera module connects to the Raspberry Pi through the camera connector, and the Raspberry Pi camera software is configured for the installed operating system.

Why a Raspberry Pi and camera rather than relying only on ultrasonic sensors:

- Ultrasonic sensors provide distance information but cannot provide the same level of visual information about an object.
- A camera can distinguish visual features such as shape, colour, edges and position.
- OpenCV provides established image-processing functions without requiring a complete computer-vision system to be developed from scratch.
- The Raspberry Pi has substantially greater computational capability than a microcontroller such as the ESP32, making it more suitable for image processing.
- A conventional microcontroller alone would have limited resources for running relatively complex computer-vision algorithms.

The camera therefore complements the ultrasonic sensors: the sensors determine physical proximity, while computer vision provides visual information about the environment.

3.13 Failure points and pre-power-on checks

Failure point	Consequence	Mitigation
5 V ECHO connected directly to a 3.3 V GPIO	Damage to the ESP32 input	1 k Ω / 2 k Ω divider on every ECHO line (§3.8)
Servo powered from a GPIO pin	Brown-out or GPIO damage when the servo loads up	Servo VCC taken only from the buck converter (§3.10)
Missing or partial common ground	Control signals not correctly referenced; erratic behaviour	Single ground node shared by every device (§3.2)
Motor driver under-rated for the motor	Overheating or driver failure	Verify the module's voltage and current ratings before connecting (§3.11)

Failure point	Consequence	Mitigation
Buck output outside tolerance	Servo and sensors misbehave or are damaged	Measure the 5 V rail on the bench before any load is connected
Motor transients on the logic rail	ESP32 resets mid-run	Motor supply taken directly from the battery, not through the buck (§3.1)
Reversed battery polarity	Immediate damage to multiple modules	Check polarity at every connector before the first power-up

Before powering the complete system, verify polarity, the common ground, the regulator output voltage, the motor-driver ratings, the servo current requirement and the 3.3 V limitation of the ESP32 GPIO inputs. Test the motor, the servo and each ultrasonic sensor separately before running the complete obstacle-avoidance program.

4. Software architecture and obstacle strategy

Rubric criterion 3 This section addresses rubric criterion 3: code modularity, state machines and flow logic, obstacle strategy, explanation of algorithms and code documentation.

4.1 Master–slave architecture

Using a Raspberry Pi as the master and an ESP32 as the slave suits an obstacle-avoiding robot because the two devices have different strengths.

The Raspberry Pi handles high-level vision

- The Raspberry Pi is powerful enough to run OpenCV and process camera images.
- It can identify obstacles or coloured objects such as red and green.
- This avoids putting computationally expensive image-processing tasks on the ESP32.

The ESP32 handles real-time control

- The ESP32 is well suited to directly controlling motors, motor drivers, ultrasonic sensors and encoders.
- It has a much more predictable real-time response than a Raspberry Pi running a full operating system.
- Once the Pi sends a command such as `dodgeRight()` or `dodgeLeft()`, the ESP32 can execute the corresponding movement immediately.

Consequences of the split

Property	Why the split delivers it
Reduced processing load	Image processing can consume considerable CPU resources. If the ESP32 were responsible for both camera processing and motor control, its resources would be more constrained. Separating the tasks lets each controller do the job it is best suited for.
Fast, reliable motor response	The ESP32 can continuously monitor sensors and control the motors without waiting for the Raspberry Pi to finish image processing — particularly useful when the robot must react quickly.
Modularity	The vision system and the motor-control system can be developed independently. The Pi could later use a more sophisticated object-detection model without major changes to the ESP32 software.
Fault isolation	If the camera-processing program encounters an error, the ESP32 still maintains control of the motors and sensors. Equally, changes to the motor-control code do not require modifying the vision system.

The architecture is therefore appropriate because it separates high-level computational tasks from low-level real-time control. The Raspberry Pi provides the processing power required for camera-based

obstacle detection and decision-making, while the ESP32 provides fast, deterministic control of the robot's actuators. Communication between them is kept simple by transmitting short commands. This makes the robot more responsive, modular, scalable and computationally efficient than relying on either controller alone.

4.2 Module decomposition

Each responsibility is implemented as a separate module so that it can be tested on its own. The table below is the intended decomposition; module names in the repository should match this table.

Module	Host	Input	Output
Frame acquisition	Raspberry Pi	Camera module	Raw frame
Image pre-processing	Raspberry Pi	Raw frame	Resized, filtered, colour-converted frame
Obstacle / colour detection	Raspberry Pi	Pre-processed frame	Detected regions and their positions
Decision and command encoding	Raspberry Pi	Detected regions	Command string
Serial transmitter	Raspberry Pi	Command string	Bytes on the UART
Serial receiver and parser	ESP32	Bytes on the UART	Validated command
Ultrasonic driver	ESP32	TRIG/ECHO lines	Three distances in cm
Clearance arbitration	ESP32	Command + three distances	Selected manoeuvre and target speed
Steering output	ESP32	Selected manoeuvre	Servo angle
Drive output	ESP32	Target speed and direction	IN1 / IN2 levels and ENA duty cycle

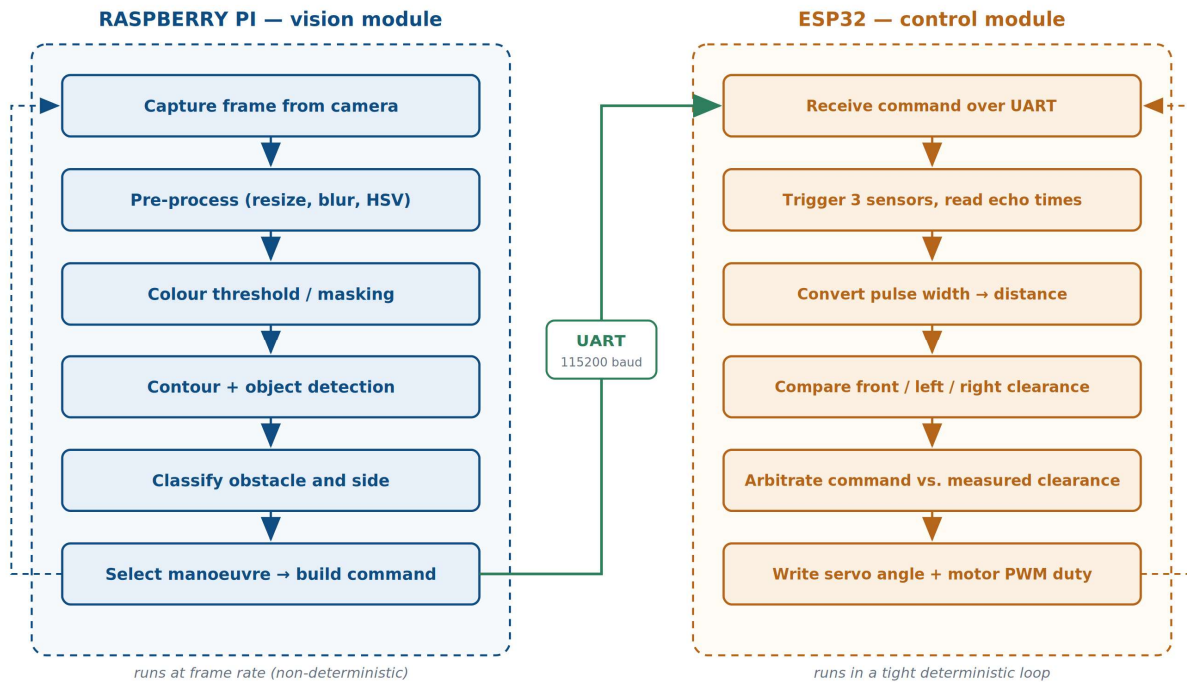


Figure 6. Software flow. The Pi pipeline runs at frame rate; the ESP32 loop runs continuously and independently of it.

4.3 Vision pipeline

The vision subsystem runs on the Raspberry Pi and follows a fixed sequence: capture, pre-process, detect, decide, transmit.

```

Camera → image capture → OpenCV processing → obstacle / colour detection
      → high-level decision → "dodgeRight()" / "dodgeLeft()" → UART → ESP32
  
```

OpenCV is used for image pre-processing, colour detection, contour and object detection, and the other vision operations required. The exact algorithm depends on the type of obstacle or colour that must be detected.

Why OpenCV rather than writing the image processing from scratch:

- It provides established functions for image acquisition, filtering, thresholding, edge detection, contour detection and object detection.
- It reduces development time.
- It is open-source and widely supported.
- It is compatible with the Raspberry Pi and with both Python and C++ environments.
- Using a software library makes the vision subsystem easier to modify and test.

Dedicated computer-vision hardware could reduce the processing load on the Raspberry Pi, but would add hardware complexity and cost.

4.4 Command protocol

A serial/UART connection transfers commands from the Raspberry Pi to the ESP32. The Raspberry Pi sends a command after processing the camera image; the ESP32 receives it and calls the corresponding control routine.

Command	Meaning	ESP32 action
FORWARD	Path is clear	Hold the steering centred and drive at the cruise duty cycle
LEFT	Obstacle should be passed on the left — <code>dodgeLeft()</code>	Steer toward the left at reduced speed, subject to measured clearance
RIGHT	Obstacle should be passed on the right — <code>dodgeRight()</code>	Steer toward the right at reduced speed, subject to measured clearance
STOP	Motion must cease	Set drive duty to zero and hold position

If physical UART pins are used, the TX of the transmitting device connects to the RX of the receiving device and vice versa, with a common ground. The voltage levels and UART configuration of the specific Raspberry Pi and ESP32 setup must be confirmed before wiring.

4.5 Obstacle-avoidance logic

The three ultrasonic sensors provide spatial information around the robot, so the ESP32 can determine not only that an obstacle is present but where the free path is.

Scenario	Left	Front	Right	Decision
Obstacle on the left	15 cm	20 cm	80 cm	The right side has substantially more free space, so a right-hand avoidance manoeuvre is selected
Obstacle on the right	80 cm	20 cm	15 cm	The left side has more clearance, so a left-hand manoeuvre is selected
Obstacle directly ahead	60 cm	15 cm	55 cm	The obstacle is primarily in the forward path; the left and right distances are compared and the clearer direction is selected

A single ultrasonic sensor would report only that something is roughly 20 cm away. Three sensors report that something is 20 cm ahead, that there is 15 cm of clearance on the left and 70 cm on the right — which is what makes a useful steering decision possible.

4.6 State machine

The behaviour described in Sections 2.5 and 4.5 is implemented on the ESP32 as an explicit state machine. Expressing it this way keeps speed and steering decisions in one place and makes the transitions testable one at a time.

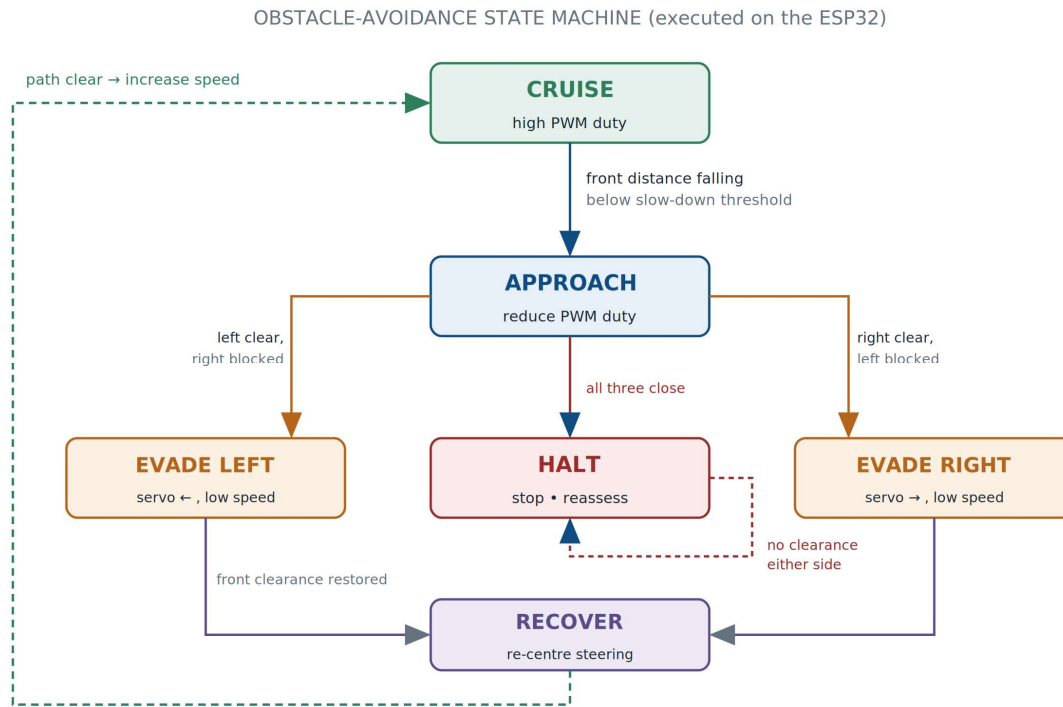


Figure 7. Obstacle-avoidance state machine executed on the ESP32.

State	Entered when	Actuator outputs	Leaves for
CRUISE	All three clearances are large	Steering centred; high PWM duty	APPROACH when the front distance falls below the slow-down threshold
APPROACH	Front distance below the slow-down threshold	Steering centred; reduced PWM duty	EVADE LEFT, EVADE RIGHT or HALT once the side clearances are compared
EVADE LEFT	Front blocked, left clearance greater	Servo to the left limit; low PWM duty	RECOVER when front clearance is restored; HALT if the left closes up
EVADE RIGHT	Front blocked, right clearance greater	Servo to the right limit; low PWM duty	RECOVER when front clearance is restored; HALT if the right closes up
HALT	All three clearances small, or no escape direction	Drive duty zero; steering held	APPROACH or CRUISE once a clearance reopens
RECOVER	The obstacle has been passed	Steering returned to centre; duty increasing	CRUISE once the path ahead is clear

4.7 Arbitration between vision and ultrasonic data

Both controllers can propose a manoeuvre, so the rule for resolving disagreement is stated explicitly. The Raspberry Pi command sets the preferred direction, because vision can distinguish what the obstacle is and can see it earlier. The measured ultrasonic clearance holds veto authority over that

preference, because it is an immediate physical measurement taken by the controller that is actually driving the actuators, and it remains valid when the vision pipeline is slow, wrong or stopped.

Rule In practice: if the Raspberry Pi commands RIGHT but the right-hand sensor reports insufficient clearance, the ESP32 does not execute the turn. It evades to the clearer side, or halts if neither side is clear.

► *Confirm that this matches the arbitration implemented in the firmware, and adjust the wording if the code gives priority differently.*

4.8 Edge cases

Edge case	Why it happens	Handling
No echo received within the timeout	The target is beyond range, angled away, or sound-absorbing	Treat the reading as ‘at least maximum range’ rather than as zero, and continue on the remaining sensors
Vision command conflicts with measured clearance	The camera sees an obstacle the sensors do not, or vice versa	Measured clearance wins (§4.7)
All three clearances small	The robot has entered a dead end or a corner	Enter HALT and reassess rather than committing to a turn that cannot complete
Serial link silent	The Pi is busy, still booting, or the vision program has stopped	The ESP32 continues to run on ultrasonic data alone — the fault-isolation property described in §4.1
Partial or corrupted command received	A byte is lost or the message is split across reads	Discard the fragment and retain the previous validated command
Obstacle detected while already evading	A second obstacle appears mid-manoeuvre	Re-evaluate the clearances on each control cycle rather than running an evade manoeuvre open-loop

4.9 Testing, tuning and performance metrics

Each part of the strategy is validated with a measurement rather than by observation alone.

Test	Method	Metric	Result
Sensor accuracy	Flat target at known distances (§3.7)	Error in cm at each distance; spread across repeats	
Stopping distance	Drive at each PWM duty toward a wall and command a stop	Distance travelled after the stop command, per duty cycle	
Threshold validation	Approach an obstacle at cruise speed	Whether the robot stops or evades before contact, over repeated runs	

Test	Method	Metric	Result
Evade success rate	Repeated runs against obstacles placed left, right and centre	Successful avoidances out of total attempts	
Control loop rate	Time the ESP32 main loop	Loop iterations per second	
End-to-end latency	Timestamp from frame capture to servo movement	Milliseconds from camera to actuator	
Turn clearance	Evade manoeuvre beside a wall	Minimum lateral gap maintained during the turn	

► Record the measured results and, where a value led to a change, note the change in the iteration log in Section 5.6.

5. Systems thinking and engineering decisions

Rubric criterion 4 This section addresses rubric criterion 4: how the subsystems work together, engineering reasoning for decisions, constraints and trade-offs, iteration and testing cycles, and risk identification and mitigation.

5.1 How the subsystems work together

The robot uses two complementary methods of obstacle detection. The ultrasonic sensors provide distance information from the front, left and right sides, while the Raspberry Pi camera provides visual information. OpenCV processes the camera images to detect obstacles. The Raspberry Pi then communicates the required direction, such as `dodgeRight()` or `dodgeLeft()`, to the ESP32. The ESP32 controls the motor driver and the servo motor to execute the avoidance manoeuvre.

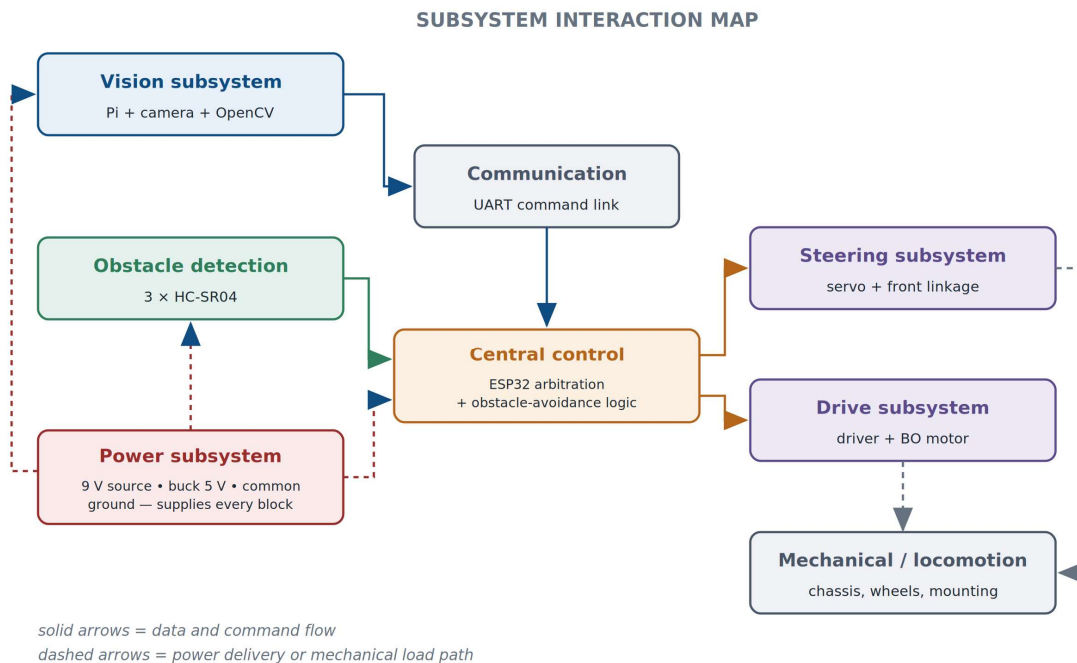


Figure 8. Subsystem interaction map. Solid arrows carry data and commands; dashed arrows carry power or mechanical load.

5.2 Why the ESP32 is the low-level controller

The ESP32 acts as the low-level controller responsible for interacting with the robot's motors, servo and sensors. It receives directions from the Raspberry Pi and executes the corresponding manoeuvre.

Why not let the Raspberry Pi control everything directly:

- The ESP32 is well suited to real-time control of motors, servos and sensors.
- It has the GPIO, PWM and communication interfaces these components require.

- Controlling motors and reading sensors is far less computationally demanding than what the Raspberry Pi is needed for.
- Separating high-level vision from low-level motor control prevents image-processing tasks from being responsible for precise motor timing.
- The ESP32 can keep handling time-critical hardware operations while the Raspberry Pi performs heavier vision processing.
- It is inexpensive and has low power consumption.

5.3 Why a master–slave split rather than a single controller

- The Raspberry Pi is better suited to computationally intensive image processing.
- The ESP32 is better suited to direct hardware control.
- Splitting the tasks reduces the computational burden on either device.
- The architecture is modular: the vision system and the motor-control system can be developed and tested separately.
- Hardware-level motor control stays on the ESP32 rather than depending on the operating-system timing of the Raspberry Pi.

This separation also makes it easier to replace or modify one subsystem without redesigning the entire robot.

5.4 Complementary sensing

The robot does not depend on a single sensing technique. It combines ultrasonic distance sensing with camera-based computer vision, because the two techniques have different strengths.

Technique	Main strength	Main limitation
Ultrasonic sensing	Direct distance measurement	Limited information about the appearance of the object
Camera and OpenCV	Provides visual information such as shape, colour and edges	Affected by lighting and requires more processing
Combined system	Complementary information and greater robustness	Greater hardware and software complexity

The camera provides visual information that ultrasonic sensors cannot, while ultrasonic sensors provide direct proximity information that does not depend on image interpretation. The detected information is converted into decisions such as `dodgeRight()` or `dodgeLeft()`, which the ESP32 then translates into steering and motor actions.

5.5 Constraints and how the design responds to them

Constraint	Design response
ESP32 GPIO inputs are 3.3 V logic; HC-SR04 ECHO is approximately 5 V	A 1 k Ω / 2 k Ω divider on each ECHO line (§3.8)
GPIO 34 and GPIO 35 are input-only	They are assigned to ECHO inputs only, and are never used for TRIG or actuator outputs (§3.9)
An ESP32 GPIO cannot supply motor current	A motor driver stands between the controller and the BO motor (§2.3)
The servo draws high current when steering against resistance	The servo is powered from the buck converter, never from a GPIO (§3.10)
A single 9 V source must serve motor, logic and actuators	The motor sits on the raw battery rail; a buck converter creates a separate 5 V rail (§3.1)
The servo has a limited angular range	Front-wheel steering is used, which needs limited travel rather than continuous rotation (§2.4)
Camera performance depends on lighting	Ultrasonic sensing provides a light-independent measurement layer (§5.4)
A full operating system gives non-deterministic timing	All time-critical actuation is executed on the ESP32 (§5.2)
Cost and complexity must stay proportionate to a small mobile robot	Ultrasonic rather than LiDAR; a standard motor driver rather than a specialised controller; a software library rather than dedicated vision hardware

5.6 Trade-off register

The table records the alternatives that were considered for each significant decision, and the reason for the option chosen.

Decision	Alternatives considered	Why the chosen option
Motor driver between the ESP32 and the BO motor	Driving the motor directly from a GPIO	A GPIO cannot supply motor current and could be damaged; the driver separates the high-current circuit and enables both PWM speed control and direction control
Front-wheel steering with a servo	Differential steering; a DC motor with an encoder for feedback	Precise angular positioning without an additional sensor, smoother directional change, simpler to interface, and only a limited angular range is needed
Buck converter for the 5 V rail	Linear regulator; a second battery	Substantially less power dissipated as heat, and no additional battery to carry; efficiency directly affects operating time

Decision	Alternatives considered	Why the chosen option
Three ultrasonic sensors	One forward sensor; infrared sensors; LiDAR	Three sensors reveal where the free path is, not just that an obstacle exists; infrared is more affected by surface reflectivity; LiDAR is more expensive than the task requires
Camera and OpenCV on the Raspberry Pi	Ultrasonic sensing only; writing the vision code from scratch; dedicated vision hardware	Visual features such as shape, colour and edges become available; an established library shortens development; dedicated hardware would add cost and complexity
Raspberry Pi master with ESP32 slave	A single controller — either the Pi alone or the ESP32 alone	Deterministic actuation on the microcontroller, heavy vision on the Pi, independent development of each side, and fault isolation between them

5.7 Risk register

Electrical failure points are listed in Section 3.13. The risks below are system-level: they concern how the robot behaves rather than whether the hardware survives.

Risk	Effect if unmitigated	Mitigation
Lighting changes at the competition venue	Colour thresholds tuned elsewhere stop detecting obstacles	The ultrasonic layer is independent of light and continues to protect the robot (§5.4); thresholds re-checked on site
The vision program stops or crashes	The robot would be left with no obstacle avoidance	The ESP32 keeps sensing and driving on its own data (§4.1, §4.8)
Timing jitter from the Pi's operating system	Late or uneven actuator updates	All time-critical control runs on the ESP32 (§5.2)
Ultrasonic reading lost on an angled or soft surface	A missing echo is misread as an obstacle at zero distance	Out-of-range readings are treated as maximum range, not zero (§4.8)
Torque falls away during a turn	The robot stalls mid-manoeuve	Speed is reduced before and during the turn to keep torque margin (§2.5)
Battery voltage sags over a run	Motor speed and servo response drift as the battery discharges	Check performance on a low battery as well as a fresh one; record it in the test results (§4.9)

5.8 Iteration and testing cycles

Design changes made in response to test results are recorded here, so the reasoning behind the final configuration remains visible.

► Each row should tie back to a measurement in Section 4.9 — the strongest evidence for this criterion is a change that was made because of data rather than because of a hunch.

6. Reproducibility and repository

Rubric criterion 5 This section addresses rubric criterion 5: repository structure and clarity, commit history, README structure, file organisation, inclusion of CAD, wiring and code files, and whether another team could reproduce the robot.

6.1 Repository structure

```

/
├── README.md           ← main documentation, ≥ 5000 characters
├── LICENSE
├── docs/
│   ├── engineering-documentation.pdf
│   ├── testing/        ← measured results, logs, tuning notes
│   └── decisions/      ← trade-off and iteration records
├── hardware/
│   ├── schematics/    ← wiring diagrams and pin tables
│   ├── cad/           ← chassis and mount source files + exports
│   └── bom             ← bill of materials
├── src/
│   ├── esp32/         ← firmware: sensors, servo, motor, serial parser
│   └── raspberry-pi/  ← vision pipeline and command transmitter
├── media/
│   ├── photos/        ← robot from every side
│   └── video/         ← or a link to the demonstration run
└── t-photos/ v-photos/ ← team and vehicle photos, if required by the season

```

6.2 What each area must contain

Area	Required contents
README.md	At least 5000 characters. Overview of the robot, subsystem summary, hardware list, wiring summary, build instructions, how to run the software, and links into docs/
docs/	This engineering documentation, plus the measured test results referenced from Sections 3.7 and 4.9
hardware/	Wiring diagrams, the pin assignment table, CAD or mechanical drawings of the chassis and mounts, and the bill of materials
src/esp32/	Firmware, with the pin definitions, the ultrasonic driver, the arbitration logic and the state machine each in their own file
src/raspberry-pi/	The vision pipeline and the serial transmitter, with any tuning parameters kept in one place rather than scattered through the code
media/	Photographs of the assembled robot from each side, and a demonstration video or a link to one

6.3 Versioning and release notes

Each meaningful change to the robot or its documentation is recorded here and reflected in a tagged release, so that a judge or another team can tell which version of the repository matches the robot as presented.

Version	Changes
V1.0	Initial engineering documentation
V2.0	Doing laps without obstacles
V3.0	Fully working bot

6.4 Repository quality checklist

	Requirement	Status
1	README is at least 5000 characters and describes the whole system	[]
2	At least three commits, with meaningful messages describing what changed and why	[]
3	Code files present for both the ESP32 and the Raspberry Pi	[]
4	Wiring diagrams present and readable	[]
5	CAD or mechanical drawing files present	[]
6	Bill of materials present	[]
7	Testing workflow documented with measured results	[]
8	Versioning or release notes included	[]
9	Another team could rebuild the robot from the repository alone	[]

6.5 Build and bring-up procedure

Following this order means that a fault is found while only one subsystem is connected, rather than after everything is powered at once.

1. Assemble the chassis, mount the front steering linkage and fit the rear drive.
2. Wire the common ground first, before any supply rail is connected.
3. Power the buck converter alone and measure its output; confirm it is 5 V before connecting any load.
4. Build the three ECHO voltage dividers and measure each output while the sensor is triggered; confirm approximately 3.3 V.
5. Flash the ESP32 and test the BO motor on its own — direction and PWM speed range.

6. Test the servo on its own across its full commanded range, checking the steering linkage does not bind at either limit.
7. Test each ultrasonic sensor separately against a known distance, and record the calibration values from Section 3.7.
8. Connect the Raspberry Pi and confirm that commands sent over the UART are received and parsed correctly, with the motors disabled.
9. Enable the motors and run the complete obstacle-avoidance program, starting at a low cruise duty cycle.
10. Record the measurements listed in Section 4.9 and log any resulting changes in Section 5.8.

Appendix A — Wiring quick reference

A single-page build sheet. Every connection in this appendix is described with its reasoning in Section 3.

A.1 Ultrasonic sensors

FRONT SENSOR	LEFT SENSOR	RIGHT SENSOR
VCC → 5 V (buck)	VCC → 5 V (buck)	VCC → 5 V (buck)
GND → common ground	GND → common ground	GND → common ground
TRIG → ESP32 GPIO 13	TRIG → ESP32 GPIO 14	TRIG → ESP32 GPIO 25
ECHO → divider → GPIO 34	ECHO → divider → GPIO 35	ECHO → divider → GPIO 32

Each divider: ECHO → 1 kΩ → tap to the ESP32 GPIO → 2 kΩ → GND. Never connect a 5 V ECHO line directly to a GPIO.

A.2 Servo

Buck +5 V	→	Servo VCC	
Buck GND	→	Servo GND	
ESP32 GPIO	→	Servo SIGNAL	(record the GPIO used)

A.3 Motor driver and BO motor

ESP32 GPIO 18	→	IN1
ESP32 GPIO 19	→	IN2
ESP32 GPIO 15	→	ENA / PWM
ESP32 GND	→	Driver GND
9 V battery	→	Driver motor-power input
Driver OUT1	→	BO motor terminal 1
Driver OUT2	→	BO motor terminal 2

A.4 Power and ground

9 V battery	→	Motor driver motor-supply input
	→	Buck converter (9 V → 5 V)
	→	Servo VCC
	→	Ultrasonic VCC × 3
	→	ESP32 5 V / VIN (verify the board first)

Common ground: battery · motor driver · buck · ESP32 · servo · ultrasonic

